Advanced Java

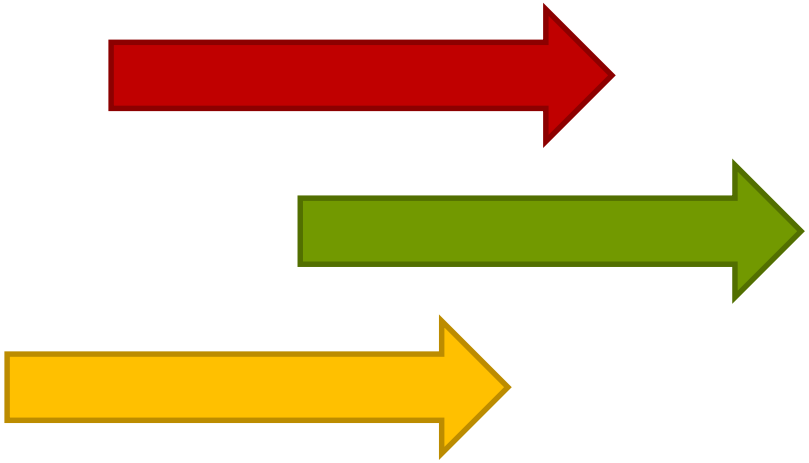
Concurrent programming

Topics

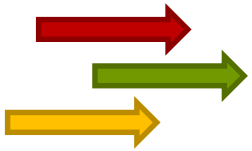
- Introduction to concurrency
- Execution model
- Memory model implications
- Practical basics
- Design concerns
- Multithreading support
- Building blocks
- Concurrent troubles

About concurrency

- Concurrency and parallelism
- Terminology
- Motivations and dangers



Concurrency × parallelism



- Concurrent actions
  - Existing within overlapping time frames
- Parallel actions
  - Being executed in the same time (and thus being concurrent)

↳ Parallelism is just one way of concurrency

- However, probably the most complex case, bringing its own problems and challenges

Why to care at all?

- Performance gains
  - Due to physical limits for increasing hardware speed, parallelism must be exploited to get more computation power
- Support for natural concurrency
  - Users require multiple operations in progress at the same time (e.g., UI should be able to react independently on other proceeding actions)
- More powerful and easier-to-use techniques
  - E.g., threads, coroutines, iterators etc.

A few definitions

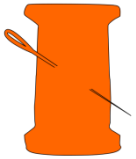
- Concurrent programming needs & brings appropriate abstractions

- Thread (of execution)
  - The smallest sequence of instructions that can be managed independently by a (system) scheduler and executed independently as well
  - It is an active entity, unlike a process, it does actually “run”

- Process
  - An instance of a program that is being executed
  - A collection of various resources (including memory) and a container for threads
  - It’s a passive entity, a process without threads would do nothing

- Shared resource
  - A resource (an entity required for a thread to run) accessible to multiple processes, or rather to multiple threads

Why to bother with threads?



- Multithreading is built in the very core of Java and its runtime environment
- Knowledge of multithreading and concurrency is implicitly assumed
- Good coding and design practices must count with that
- Surprisingly, a lot of developers lack deep understanding of the essentials… which leads sometimes to catastrophic failures

Pitfalls and dangers

- Concurrency threatens (naïve) consistency
  - Care and some guarantees are requires to achieve the desired (formal) consistency
  - Programmers have to understand the essentials very well
- Concurrent actions result in unpredictable outcomes
  - However, if desired consistency is kept, the outcomes can still be valid and reasonable
  - Anyway, excluding the possibility of invalid outcomes is a challenge
- Bugs in concurrent code are very difficult to find
  - Invalid outcomes can appear rarely or depend on particular platforms (even in Java)

Discovering the parallel reality

- Heading to the core of parallelism
- Real hardware considerations

Sharing a resource (or a state)

Thread A

```
int amount = …
balance = balance + amount;
```

```
int balance;
```

A shared piece of memory

Thread B

```
int amount = …
balance = balance + amount;
```

- When sharing a resource…
  …multiple threads may access it
- When touching it concurrently
  …a conflict may occur
- When a conflict occurs…
  …the outcome is indeterminate
- Preventing concurrent access prevents the conflict and prevents threads from ruining the balance
- That's why locks were invented
  - To prevent unwanted access ;-)
  - To make an action appear atomic

About concurrent programming

"Perhaps surprisingly, concurrent programming isn't so much about threads or locks, any more than civil engineering is about rivets and I-beams.

Writing thread-safe code is, at its core, about managing access to state, and in particular to shared, mutable state."

Brian Goetz

- Alright, this **is** important, but there is something more yet
  - We'll have to deal with threads and locks for a while to find it out

Let's write a lock…
A very, very simple lock
Can you?

Ehm…

A very simple lock

```
Lock::lock

int tid = currentThreadId();
while (!OWNER.compareAndSet(this, 0, tid)) {}
```

```
int owner;
```

A shared piece of memory – the instance of `Lock`

```
Lock::unlock

int tid = currentThreadId();
if (!OWNER.compareAndSet(this, tid, 0)) {
    throw new Error(…);
}
```

The `compareAndSet` operation must compare a value in a variable and set it to a given value **atomically**

That's cheating a bit, I admit… but:

- such an operation is supported by the hardware usually (so called CAS instructions)
- there are solutions that can work without that (Peterson's algorithm, Dekker's algorithm), but still with some assumptions about the hardware

Sharing… revisited

Thread A

```
int amount = …
L.lock();
balance = balance + amount;
L.unlock();
```

```
int balance;
Lock L;
```

A shared piece of memory

Thread B

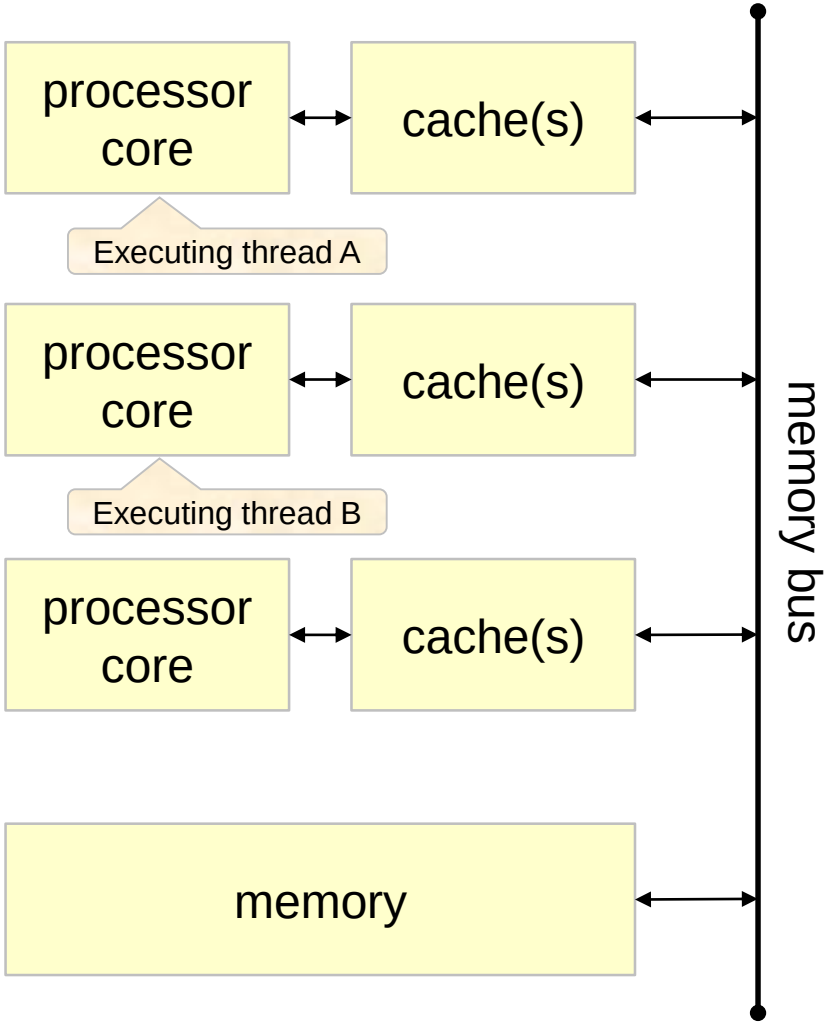
```
int amount = …
L.lock();
balance = balance + amount;
L.unlock();
```

If the lock works, the conflict is avoided…

…so we are safe, right?

Right?

About memory architecture



We would certainly be safe…

- if there were a single CPU core
- or if all (?) memory operations were synchronous
- and the CPU(s) were somewhat simple (which needs a lengthy explanation)

Otherwise thread A may **see** a different value at the same memory address than thread B (because of caching… mostly)

↳ We have to consider memory **visibility** for particular threads

Having all memory access synchronous is too harsh and inefficient, but it might be sufficient to synchronize just something…

About real hardware

- Hardware must face difficult compromises
  - Memory transactions depend on cache lines and bus width
  - Memory transactions become even more challenging with growing number of attached components… like cores
  - Word width and bus width may differ (which may be one of the reasons for word tearing)
  - No particular outcome guaranteed with data races
- Hardware attempts peculiar things
  - Instructions may be executed out of the original order, in parallel or even speculatively
- There are some physical limits
  - How far light reaches within a tick of a CPU clock?

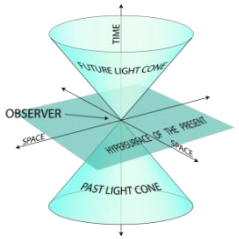
Time perspective

Access	Size	Latency [ns]	Latency scaled for humans
Register	1–32 B	0.3	1 s
L1 cache	~ 32 KB	1.2	4 s
L2 cache	~ 256 KB	3	10 s
L3 cache	~ 8 MB	12	40 s
Local RAM		60	3 min 20 s
Remote RAM		100	5 min 33 s
SSD I/O		~ 50 000 (50 µs)	46 h 17 min
HDD I/O (magnetic)		~ 4 000 000 (4 ms)	~ 155 days

Execution model

- Concurrency basic rules
- Defining a model
- Trade-offs

Concurrency basic rules



- **All actions must be considered with the respect to their mutual order**, i.e., with the respect to their executors and the observers
- **The visibility of the outcomes is essential**

↳ **We need to ensure a well-defined ordering of all actions and the visibility of their outcomes**

"What *happens before* this action and who can see the results?"

Execution model in general

- Syntax of a programming language is not interesting for the time being: it tells how to write the source, but not how the code executes
- Semantics of a programming language is now important: it is described via the semantics of the abstract machine executing the source program
- Significant part of the machine specifications are the rules concerning how memory storage behaves: the memory model
- A memory model must answer a simple question: **what values can a particular *read* in a program return?**

Execution model and Java

- Java is an imperative language (the code implies some kind of direct execution)
- Java specification sticks to natural code reading (top-down, left-to-right)

➪ Execution model seems (locally) sequential

✓ Java is very programmer-friendly and covers infamous gaps known, e.g., from C/C++

? However, misunderstanding can come easily, e.g., as the result of a simplistic interpretation

Memory model trade-offs

- Memory model must consider:
  - How difficult is using the language
  - How difficult is implementing the language
  - How difficult is building appropriate hardware

- Java Memory Model deserves highest credit
  - Providing quite complete formal specification
  - Taking into account various platforms, while retaining all the portability
  - Still sane enough, so that the programmers would not go crazy

Quiz

Let’s have a look…

Correct or incorrect?

```
public final class Box<T> implements Supplier<T> {

    private T value;

    public Box() {
        // Default constructor
    }

    public Box(T argument) {
        value = argument;
    }

    public T get() {
        return value;
    }

    public synchronized void set(T argument) {
        value = argument;
    }
}
```

And what about now?

```
public final class Box<T> implements Supplier<T> {

    private T value;

    public Box() {
        // Default constructor
    }

    public Box(T argument) {
        value = argument;
    }

    public synchronized T get() {
        return value;
    }

    public synchronized void set(T argument) {
        value = argument;
    }
}
```

Wait. Is it really necessary? It was **synchronized**, *wasn't it?*

What should be better here?

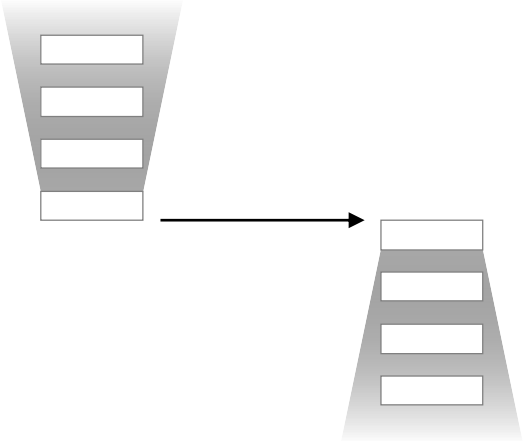
```
public final class DoublingSupplier
    implements IntSupplier {

    private int value;

    public DoublingSupplier(int parameter) {
        value = parameter * 2;
    }

    public int getAsInt() {
        return value;
    }
}
```

This field should have been `final`.

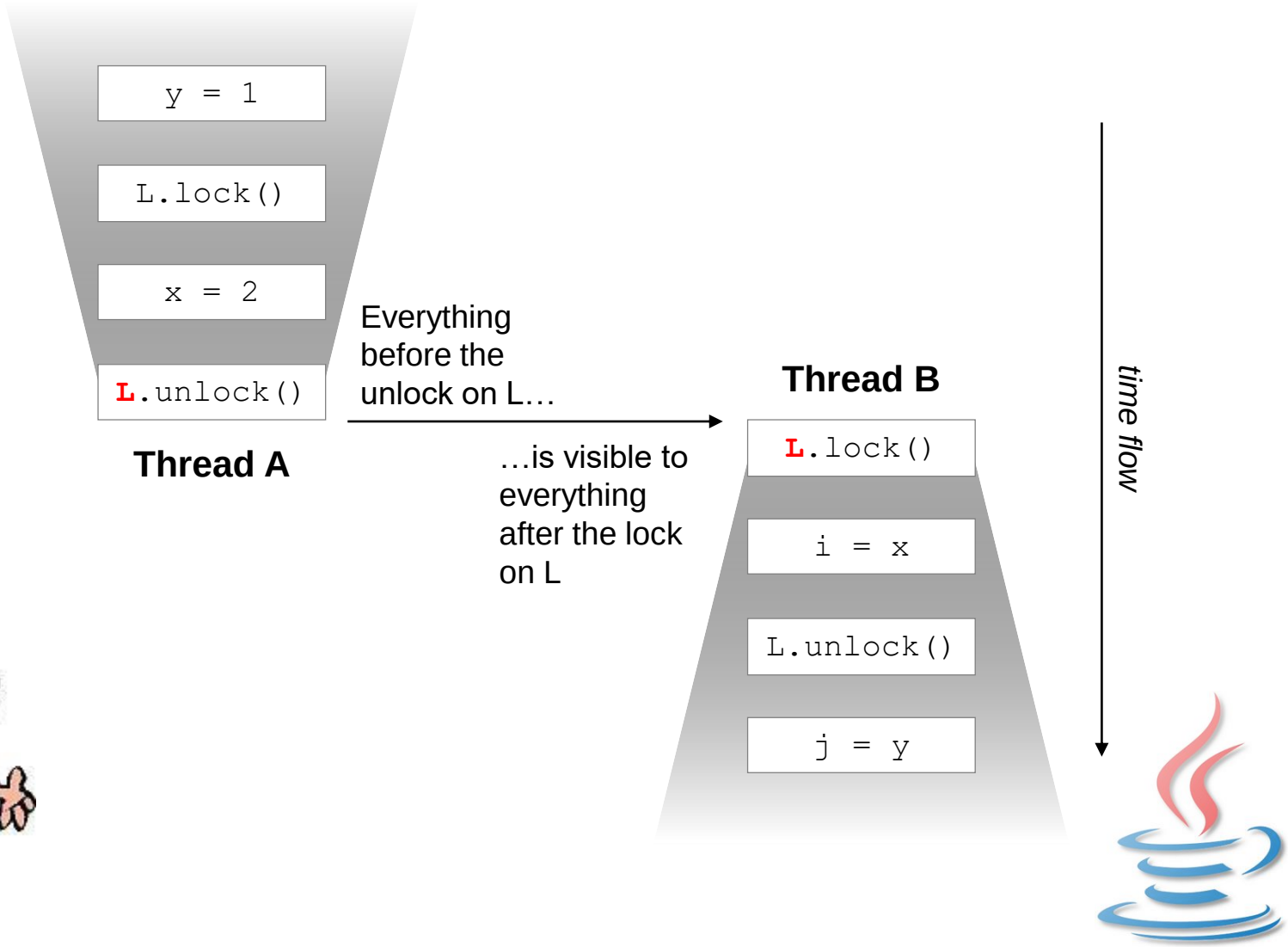
Come on. Writing `final` is annoying. It's useless anyway.

Java Memory Model

- Definition and rules
- Consequences
- Techniques



Java Memory Model in a picture



Java Memory Model in 8 minutes

- Java Memory Model (JMM) defines a **partial order** called *happens-before*:

  "If action A *happens before* action B, then action B can see the results of action A (whether or not A and B occur in different threads)"

- The (basic) rules for the "happens-before" order:
  - **Program order rule**: each action in a thread happens before every action in that thread that comes later in the program order
  - **Lock rule**: an unlock of a lock happens before every subsequent lock on that same lock (this applies on library locks as well as on intrinsic locks)
  - **Volatile variable rule**: a write to a volatile field happens before every subsequent read of that same field (this applies on atomic variables too)
  - **Thread start rule**: a request to start a thread happens before every action in the started thread
  - **Thread termination rule**: any action in a thread happens before any other thread detects that thread has terminated
  - **Interruption rule**: requesting an interruption on a thread happens before the interrupted thread detects the interruption request
  - **Finalizer rule**: the end of an object's constructor execution happens before the start of the finalizer for that object

JMM formally: the big picture

- JMM describes the allowed program outcomes by describing what *executions* are conforming
- It does so by introducing *actions*, and *orders* over actions, and *consistency rules* that govern what actions, orders and constraints constitute a valid execution
- **If a program result can be explained by some valid execution, then this result is allowed under JMM**

JMM formally: action orders

- **Program order** (PO): defines a total order of actions within each thread

- **Synchronization order** (SO): defines a total order over synchronization actions (explicitly listed in the specification)
  - **SO consistency**: all reads that come later in SO see the last write to that location
  - **SO-PO consistency**: the order of synchronization actions in SO within a single thread is consistent with PO in that thread
  - ↳ SO consistency and SO-PO consistency mean that in all conforming executions synchronized actions appear sequentially consistent

- **Synchronized-with order** (SW): a partial suborder of SO that pairs specific synchronization actions that "see" each other
  - This order serves as the connection bridge between different threads

JMM formally: happens-before

- **Happens-before order** (HB): the transitive closure of the union of PO and SW
  - A partial order, and only relates **some** actions, **not** every pair of actions
  - Able to relate the actions that are not synchronization actions, which allows us to cover all important actions with ordering guarantees
  - **HB consistency**: every read can see either the latest write in the happens-before order (notice the symmetry with SO here), or any other write that is not ordered by HB (this allows races)

JMM formally: additional rules

- **Causality rules**: additional verifications on otherwise conforming executions, to rule out causality loops

- **Final field rules**: providing additional constraints imposed by `final` fields, e.g., additional happens-before relationship between `final` field stores and their associated reads

Benefits of Java Memory Model

- Java Memory Model implies directly:
  - Locks provide both atomicity and visibility
  - Volatile variables provide just the visibility
  - Atomic variables (see later) provide the visibility and limited atomicity

- "Piggybacking" technique
  - The strength of "happens-before" allows to piggyback sometimes on the visibility guarantees that are implied by the existing "happens-before" enforcement
  - This technique is quite fragile, but can gain some performance and allows, e.g., safe using of various thread-safe collections in a natural way as a side-effect

- Reasonable price for achieving thread safety measured by the complexity and the imposed performance penalty

Stale data

- If a thread reads data from a memory place without ensuring the proper visibility guarantee, it may load *stale data* (the data which are not fresh because of a change performed by another thread in the memory place)

- What are the consequences?
  - Usually bad enough – it is the common cause of race conditions and other hazards… stale data can spoil anything

? But… there is something to know anyway: "out-of-thin-air" safety
  - ➢ This kind of safety tells that stale data can only contain the data written by a thread in the past (although not telling which thread and how old the data might be – you just know the data are not random)

  ☠ It does **not** apply on `double` and `long` values (see JVM internals)

  ± It applies on all other values (including references, but not objects!)

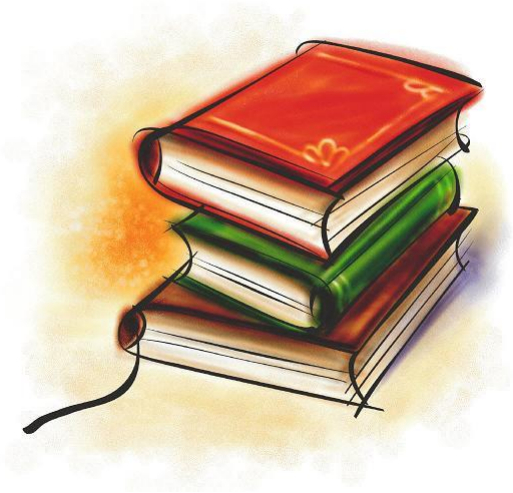
  ± This kind of safety has a use, e.g., for hash code caching technique

Often mistakes

- Coding based on anecdotal observations of what language runtimes are doing today
- Too harsh mental shortcuts, e.g., imaging barriers instead of using actual JMM guarantees
- Mixing happens-before order and physical execution order
- Using `volatile` solves all problems
- Java is portable, so when it runs correctly on my platform…

Concurrency in theory

- Safe publication, immutability
- Confinement
- Sharing policies



State management

"Writing thread-safe code is, at its core, about managing access to state, and in particular to shared, mutable state."

Brian Goetz

- We have to care about state and how it can be accessed
  - Stale data can spoil everything, out-of-thin-air safety does not save us in general
- Hence we have to answer following questions
  - When some state should or should not be visible?
  - How to achieve the conditions for safe access?
  - How to prevent unsafe access?
  - Which access policies can we actually declare?
- We have to revive the concept of ownership

Escape… and publication

- **Publishing** an object means making it available to code outside of the object's current scope
  - Publishing an object may indirectly publish others
  - Publishing can compromise encapsulation, break invariants… (e.g., due to leaking references to an object's internals)

- When an object is published when it should not have been, it is said to have **escaped**
  - It brings many hazards: abusing or misusing the object, accessing an incomplete instance (if the object escapes during its construction), viewing an inconsistent state of the object (publishing without considering memory visibility) etc.

- Never allow an object to escape, always ensure safe publication

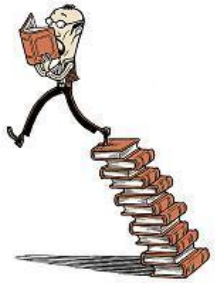
Intermezzo: immutability

- A truly **immutable** instance has just one state which can't be changed
- An **effectively immutable** instance has just one externally visible state which can't be changed

- Effective immutability is weaker
  - An effectively immutable class/instance needs more care to be really safe

- Immutability in general will be discussed later

Safe publication: considerations



Initialization safety: `final` field semantics

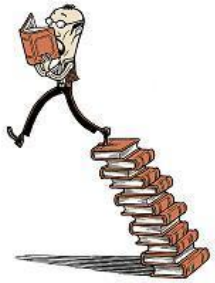
- **"A thread that can only see a reference to an object after that object has been completely initialized is guaranteed to see the correctly initialized values for that object's `final` fields"**
- This is another huge point in favor of (true) immutability and a help for other objects too (especially for achieving at least the effective immutability)

I thought that `final` was just...

What to watch carefully?

- ☠ Construction time: leaking incomplete instance
- ❗ Fields: inspect what is loaded and stored
- ❗ Return values: the most common way for an escape
- ❗ Method arguments: often not realized that they accept values
- ❗ Inner (non-static) class instances: contain the implicit reference to the outer instance

Safe publication: techniques



What is safe then?

A properly constructed object can be published by…

- Initializing the object reference in a `static` initializer
- Storing a reference to it into a `volatile` field or an atomic reference instance (e.g., `AtomicReference`)
- Storing a reference to it into a `final` field of a properly constructed object
- Storing a reference to it into a properly synchronized field
  - Either using explicit locks or using some form of piggybacking
  - Various thread-safe library collections can be included here

↳ Then reading from such a reference source provides a safely published object

Safe publication: summary

✓ Immutable objects can be published through any mechanism

± Effectively immutable objects must be safely published

😐 Mutable objects must be safely published and must be either thread-safe or guarded (see later)

❗ Always document the publishing requirement and the policy how to deal with the instances – it's a part of the contract

What?! Documentation?

Confinement: overview

Confining some data ~ preventing the data from sharing

- When an object is confined to a thread, its usage becomes thread-safe even if the confined object itself is not

- ✓ No sharing, no synchronization → this is the cheapest way
- ✓ It allows to use safely the code which is not thread-safe
- ✓ It can help to avoid deadlocks and overwhelming complexity
- ± It is just the matter of design and implementation
  - The compiler is not helpful at all

- Used very often, in many cases implicitly or “by the way”

Confinement: techniques

Technique	Description
Confining to dedicated privileged threads	Original and typical use, common for GUI frameworks, database connections…
Ad-hoc thread confinement	Delegating the execution of some single-threaded code to a single thread while sharing the data is avoided during the execution Correct memory visibility is required!
Stack confinement	The data are reachable through the local variables only → confined intrinsically
Instance confinement	Encapsulating a thread-unsafe instance to provide thread-safe access to it
Thread locals	Every thread can use the same “global” way to access a private copy of some data

Common sharing policies

- Thread confinement
  - A thread-confined object is owned exclusively by a thread
  - Only the owning thread can access the object
- Shared read-only
  - A shared read-only object can be accessed by multiple threads without additional synchronization, but can’t be modified by any thread
- Shared thread-safe
  - A thread-safe object performs synchronization internally
  - Multiple threads can freely access it through its public interface without further synchronization
- Guarded
  - A guarded object can be accessed only with a specific lock held
  - Guarded objects include those that are encapsulated within other thread-safe objects and published objects that are known to be guarded

Ownership

- When no garbage collector exists, the concept of ownership is much clearer: there is somebody responsible for destroying the instance
- But the concept of ownership remains valid even when a garbage collector takes this duty – somebody still may have privileges that the others lack (or should lack)

↳ This becomes apparent when considering thread safety – e.g., confinement manifests some kind of object ownership

➢ Ownership options: exclusive ownership, shared ownership, split ownership, ownership passing

Summary

What you should remember

Conclusion

- Actions' mutual order and visibility of their effects are essential
  - The order must be always proved using the memory model's rules
- Basic rules for *happens-before*
  - Especially "lock" and "volatile" rules
- Limit mutability, strive for true immutability – use `final`
  - Highly recommended, little known, frequently neglected
  - Limiting mutability is always a good idea anyway
- Always consider confinement
  - Often the cheapest and least complicated option
- Never allow an object to escape, publish safely + document
  - Watch out what becomes available outside an instance
  - Consider object ownership

Concurrency in practice

- Basic concurrency support
- Locks and atomic variables
- Common idioms

Concurrency support… so far

- Field types: `volatile` and `final`
  - Special semantics concerning memory visibility; `volatile` rules apply to atomic variables as well
- Locks: referred in Java Memory Model rules
  - Based on the monitor abstraction
  - Intrinsic: built-in, every object can be used as a lock when applying keyword `synchronized`
  - Explicit: implemented as classes in the runtime library
- Representation of a thread: class `Thread`

Questionnaire

- Have you experienced the situation when your code had to wait for something?
- Have you experienced the situation when your code was waiting for something and you needed/wanted to interrupt the waiting?
- In your opinion, how a blocking method shall be interrupted?
- In your opinion, what a blocking method should do when interrupted?
- In your opinion, how a blocking method should indicate an interruption?
- Have you ever met `InterruptedException`?
- How do you handle `InterruptedException`?

Blocking & interruptible methods

- ❓ A thread can't be forced to do something, it can't be even killed (except for terminating the whole JVM) – there are good reasons for that

- ➢ Java/JVM offers a cooperative mechanism for interruptions: an interruption is a way to **ask** a thread to 'stop' (something) by adjusting the thread's interruption state (see `Thread::interrupt` & friends)

- ➢ Blocking methods (usually) check (or should check) the interruption state and throw `InterruptedException` when the thread has been interrupted

- ✓ It is the most sensible handling in the general case, usual especially for library code and reusable components

Handling interruptions

- When calling such a blocking method, you have to deal with the exception

- Propagate the exception further
  - This way forces the calling method to declare `throws InterruptedException` which is not always possible (e.g., within `Runnable::run`)

- Swallow the exception and restore the interruption state of the current thread:

```
try {
    …
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}
```

Intrinsic locks

- JVM built-in support (bytecode instructions `monitorenter` and `monitorexit`)
- Core supportive methods on the `Object` class: `notify`, `notifyAll`, `wait`
- Java language uses the **`synchronized`** keyword for emitting the appropriate JVM instructions

✓ Efficient and JVM may optimize it in run time

✓ Possible to detect deadlocks with tools easily

✗ Somewhat limited and not so efficient in more complex scenarios

Explicit locks: about

- Implemented in the runtime library using existing support in JVM
- Their capabilities may vary (depending on the implementation) and it is possible to make an own implementation
- See `java.util.concurrent.locks` package for the standard offer

✓ Much more powerful

✗ Not so easy to use usually

✗ Not so efficient (at least in the simple cases)

✗ Limited tool support (JVM optimization, deadlock detection etc.)

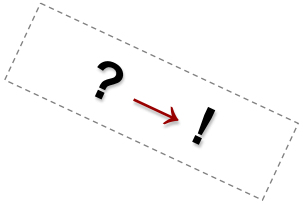
Explicit locks: explained

- Explicit locks: `Lock` (since Java 5)
  - Many new features over intrinsic locks (see the interface)

- Pros and cons of the interface
  - ✓ Timed and polled acquisition
  - ✓ Interruptible acquisition
  - ✓ Non-structured locking possible
  - ✓ Improved condition variable support (see later)
  - ✗ More difficult for JVM to optimize them or list them in thread dumps (no stack-frame relationship, harder diagnostics)
  - ☠ Easy to forget to unlock it: **always** unlock in `finally`

- The implementation further offers:
  - ✓ Read-write locking (see `ReentrantReadWriteLock`)
  - ✓ The **possibility** to use fair locking (at some cost!)
    - Both `ReentrantLock` and `ReentrantReadWriteLock`

Condition variables: theory

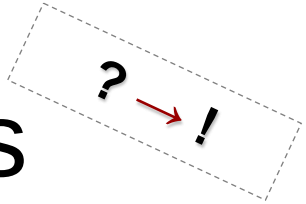


- State-based preconditions → state-dependent operations
  - In a concurrent environment it makes sense to wait for a precondition to become true, i.e., let the operation block (see, e.g., `BlockingQueue`)

- Structure of a blocking state-dependent operation:

```
? blockingAction() throws InterruptedException {
    acquire lock on object state
    while (precondition does not hold) {
        release lock
        wait until precondition might hold
        optionally fail if interrupted or timeout expires
        reacquire lock
    }

    perform action
    release lock
}
```

Condition variables: possibilities

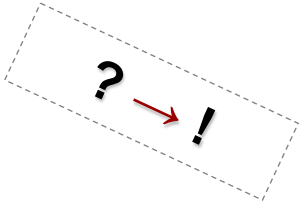


- Condition queues – how to deal with conditions variables
  - Implicit and simple: provided by intrinsic locks (see `Object`'s `wait`, `notify` and `notifyAll` methods)
  - Explicit and richer: provided by explicit locks (see `Condition` interface)

- Choosing the proper way to support conditions
  - ✗ Never ever wait in a loop with `sleep`: it's a sin
  - Use implicit condition queues when…
    - You don't need any advanced features
    - You don't have an explicit lock already
    - You have a "simple single" predicate

- Both implicit and explicit condition queues have similar rules and use, they follow the same abstraction after all

Condition variables: practice



- The condition predicate, a lock and a condition queue are associated
  - Every waiting in a condition queue is implicitly bound to the predicate
  - The association is not directly visible → document it

- Waking up does not mean the condition is (still) true
  - Another waiter could just steal the chance, or a spurious wake-up could occur

- Missed signals: some of the protocol rules was violated
  - Ensure the predicates are guarded with the appropriate lock and hold the lock when going to wait on the predicate
  - Always have a condition for waiting and test it before and after waiting
  - Always invoke waiting in a loop (helps with the above too)

- Missing notifications: was something forgotten?
  - Ensure the waiters will be woken up when the condition becomes true
  - When unsure, initiate a wake-up: e.g., when `InterruptedException` occurs
  - You may and should notify a single waiter iff both following statements hold true:
    - Uniform waiters: only one condition is associated with the condition queue, and each thread executes the same logic upon waking up
    - One-in, one-out: a notification enables at most one thread to proceed

To be continued…

References

- Java language & VM specifications
  https://docs.oracle.com/javase/specs/

- Oracle Java tutorials
  https://docs.oracle.com/javase/tutorial/essential/concurrency/

- Java SE API reference
  https://docs.oracle.com/javase/11/docs/api/

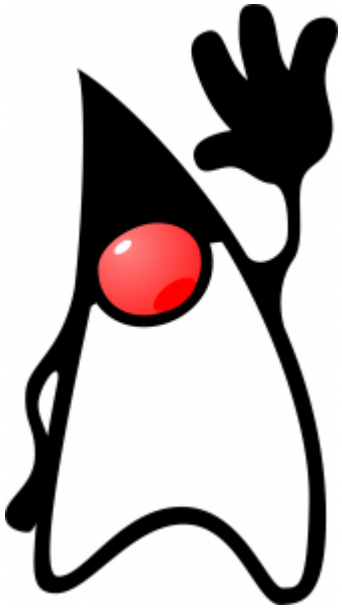
- *Java Concurrency in Practice*
  Brian Goetz

- *Aleksey Shipilëv's blog*
  https://shipilev.net/blog/2014/jmm-pragmatics/
  https://shipilev.net/blog/2016/close-encounters-of-jmm-kind/

- *Doug Lea: Using JDK 9 Memory Order Modes*
  http://gee.cs.oswego.edu/dl/html/j9mm.html

See you soon!

Feel free to contact me in between



Petr Doležal
petr.dolezal@matfyz.cz